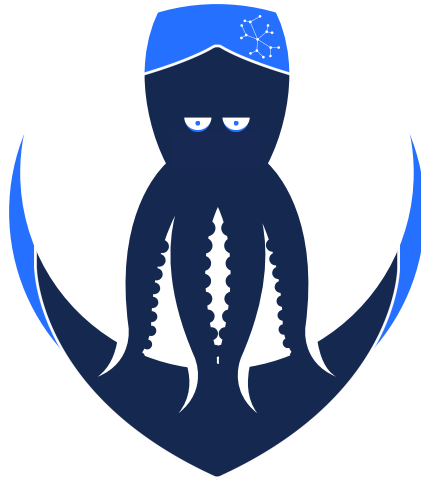




Thunder Loan Initial Audit Report

Version 0.1

@aimanubayd

September 18, 2026

Thunder Loan Audit Report

Aiman Ubayd

Sep 2, 2026

Thunder Loan Audit Report

Prepared by: Aiman Ubayd

Assisting Auditors:

- None

Table of contents

See table

- Thunder Loan Audit Report
- Table of contents
- About Aiman Ubayd
- Disclaimer
- Risk Classification
- Audit Details
 - Scope
- Protocol Summary
 - Roles
- Executive Summary

- Issues found
- Findings
 - High
 - * [H-1] Mixing up variable location causes storage collisions in `ThunderLoan::s_flashLoanFee` and `ThunderLoan::s_currentlyFlashLoaning` (Reordered storage variables between upgrades -> corrupted fee value and mapping slot after upgrade)
 - * [H-2] Unnecessary `updateExchangeRate` in `deposit` function incorrectly updates `exchangeRate` preventing withdrawals and unfairly changing reward distribution (Exchange rate updated on every deposit instead of only on fee-generating events -> inflated exchange rate blocks redemptions and misallocates yield)
 - * [H-3] By calling a flashloan and then `ThunderLoan::deposit` instead of `ThunderLoan::repay` users can steal all funds from the protocol (Flash loan repayment verified only by raw token balance, not by which function returned the funds -> attacker mints AssetTokens with borrowed funds and redeems them, draining the pool)
 - * [H-4] `getPriceOfOnePoolTokenInWeth` uses the TSwap price which doesn't account for decimals, also fee precision is 18 decimals (Price and fee math assume all tokens use 18 decimals -> incorrect fee/price calculations for non-18-decimal tokens)
 - Medium
 - * [M-1] Centralization risk for trusted owners (Single `onlyOwner` address controls token allow-listing and contract upgrades -> a malicious or compromised owner can brick redemptions or rewrite protocol logic entirely)
 - Impact:
 - Contralized owners can brick redemptions by disapproving of a specific token
 - * [M-2] Using TSwap as price oracle leads to price and oracle manipulation attacks (Flash-loanable capital used to manipulate a low-liquidity AMM's spot price -> attacker distorts ThunderLoan's fee calculation to their advantage)
 - * [M-4] Fee on transfer, rebase, etc (Protocol assumes transferred amount always equals received amount -> accounting breaks for fee-on-transfer and rebasing tokens)
 - Low
 - * [L-1] Empty Function Body - Consider commenting why (Uncommented empty override -> reviewers cannot tell whether the empty body is intentional or a missing implementation)
 - * [L-2] Initializers could be front-run (Unrestricted first-caller of `initialize` -> attacker who wins the race can set themselves as owner or point the oracle at a malicious factory)

- * [L-3] Missing critical event emissions (No event emitted when `s_flashLoanFee` changes -> off-chain systems and users cannot detect fee changes)
- Informational
 - * [I-1] Poor Test Coverage (Coverage below acceptable threshold across core contracts -> undiscovered edge cases and regressions are more likely to reach production)
 - * [I-2] Not using `__gap [ 50 ]` for future storage collision mitigation (No reserved storage slots in upgradeable base contracts -> adding new state variables to a parent contract in a future upgrade will shift and corrupt child contract storage)
 - * [I-3] Different decimals may cause confusion. ie: `AssetToken` has 18, but asset has 6 (Fixed 18-decimal assumption in `AssetToken` regardless of underlying asset's actual decimals -> confusing off-chain displays and potential integration bugs)
 - * [I-4] Doesn't follow <https://eips.ethereum.org/EIPS/eip-3156> (Custom flash loan interface instead of the standard -> reduced composability with tooling and aggregators built against EIP-3156)
- Gas
 - * [GAS-1] Using bools for storage incurs overhead
 - * [GAS-2] Using `private` rather than `public` for constants, saves gas
 - * [GAS-3] Unnecessary SLOAD when logging new exchange rate

About Aiman Ubayd

Aiman Ubayd is a smart contract auditor and security researcher. He researches how EVM & DeFi protocols fail, tracing exploit paths in protocol logic, economics, and trust boundaries. He combines manual security research with fuzzing, invariant testing, and formal verification, backed by AI-powered tooling and AI agents for web3 security analysis.

- **X (Twitter):** [<https://x.com/aimanubayd>]
- **GitHub:** [<https://github.com/aimanubayd>]
- **LinkedIn:** [<https://linkedin.com/in/aimanubayd>]
- **Contact:** [aimanubayd@gmail.com]

Disclaimer

Aiman Ubayd makes all effort to find as many vulnerabilities in the code in the given time period, but holds no responsibilities for the findings provided in this document. A security audit by the team is not

an endorsement of the underlying business or product. The audit was time-boxed and the review of the code was solely on the security aspects of the Solidity implementation of the contracts.

Risk Classification

		Impact		
		High	Medium	Low
Likelihood	High	H	H/M	M
	Medium	H/M	M	M/L
	Low	M	M/L	L

Audit Details

The findings described in this document correspond the following commit hash:

```
1 026da6e73fde0dd0a650d623d0411547e3188909
```

Scope

```
1  |-- interfaces
2  |    |-- IFlashLoanReceiver.sol
3  |    |-- IPoolFactory.sol
4  |    |-- ISwapPool.sol
5  |    |-- IThunderLoan.sol
6  |-- protocol
7  |    |-- AssetToken.sol
8  |    |-- OracleUpgradeable.sol
9  |    |-- ThunderLoan.sol
10 |-- upgradedProtocol
11    |-- ThunderLoanUpgraded.sol
```

Protocol Summary

Puppy Rafle is a protocol dedicated to raffling off puppy NFTs with varying rarities. A portion of entrance fees go to the winner, and a fee is taken by another address decided by the protocol owner.

Roles

- Owner: The owner of the protocol who has the power to upgrade the implementation.
- Liquidity Provider: A user who deposits assets into the protocol to earn interest.
- User: A user who takes out flash loans from the protocol.

Executive Summary

Issues found

Severity	Number of issues found
High	4
Medium	4
Low	3
Info	4
Gas	3
Total	18

Findings

High

[H-1] Mixing up variable location causes storage collisions in

ThunderLoan::s_flashLoanFee and ThunderLoan::s_currentlyFlashLoaning
(Reordered storage variables between upgrades -> corrupted fee value and mapping slot after upgrade)

Description: `ThunderLoan.sol` has two variables in the following order:

```
1    uint256 private s_feePrecision;  
2    uint256 private s_flashLoanFee; // 0.3% ETH fee
```

However, the expected upgraded contract `ThunderLoanUpgraded.sol` has them in a different order.

```
1    uint256 private s_flashLoanFee; // 0.3% ETH fee  
2    uint256 public constant FEE_PRECISION = 1e18;
```

Due to how Solidity storage works, after the upgrade, the `s_flashLoanFee` will have the value of `s_feePrecision`. You cannot adjust the positions of storage variables when working with upgradeable contracts.

Impact: After upgrade, the `s_flashLoanFee` will have the value of `s_feePrecision`. This means that users who take out flash loans right after an upgrade will be charged the wrong fee. Additionally the `s_currentlyFlashLoaning` mapping will start on the wrong storage slot.

Proof of Concept:

Code

Add the following code to the `ThunderLoanTest.t.sol` file.

```
1 // You'll need to import `ThunderLoanUpgraded` as well  
2 import { ThunderLoanUpgraded } from "../../src/upgradedProtocol/  
    ThunderLoanUpgraded.sol";  
3  
4 function testUpgradeBreaks() public {  
5     uint256 feeBeforeUpgrade = thunderLoan.getFee();  
6     vm.startPrank(thunderLoan.owner());  
7     ThunderLoanUpgraded upgraded = new ThunderLoanUpgraded();  
8     thunderLoan.upgradeTo(address(upgraded));  
9     uint256 feeAfterUpgrade = thunderLoan.getFee();  
10
```

```
11     assert(feeBeforeUpgrade != feeAfterUpgrade);
12 }
```

You can also see the storage layout difference by running `forge inspect ThunderLoan storage` and `forge inspect ThunderLoanUpgraded storage`

Recommended Mitigation: Do not switch the positions of the storage variables on upgrade, and leave a blank if you're going to replace a storage variable with a constant. In `ThunderLoanUpgraded.sol`:

```
1 - uint256 private s_flashLoanFee; // 0.3% ETH fee
2 - uint256 public constant FEE_PRECISION = 1e18;
3 + uint256 private s_blank;
4 + uint256 private s_flashLoanFee;
5 + uint256 public constant FEE_PRECISION = 1e18;
```

[H-2] Unnecessary updateExchangeRate in deposit function incorrectly updates exchangeRate preventing withdrawals and unfairly changing reward distribution (Exchange rate updated on every deposit instead of only on fee-generating events -> inflated exchange rate blocks redemptions and misallocates yield)

Description: In the `ThunderLoan` system, the `exchangeRate` is responsible for calculating the exchange rate between `AssetTokens` and underlying tokens. It is, in a way, responsible for keeping track of how much of the underlying token liquidity providers are owed. However, the `deposit` function updates this rate, without collecting any fees.

```
1     function deposit(IERC20 token, uint256 amount) external
2         revertIfZero(amount) revertIfNotAllowedToken(token) {
3         AssetToken assetToken = s_tokenToAssetToken[token];
4         uint256 exchangeRate = assetToken.getExchangeRate();
5         uint256 mintAmount = (amount * assetToken.
6             EXCHANGE_RATE_PRECISION()) / exchangeRate;
7         emit Deposit(msg.sender, token, amount);
8         assetToken.mint(msg.sender, mintAmount);
9         uint256 calculatedFee = getCalculatedFee(token, amount);
10        @> assetToken.updateExchangeRate(calculatedFee);
11        token.safeTransferFrom(msg.sender, address(assetToken), amount)
12        ;
13    }
```

The `updateExchangeRate` call in `deposit` treats a normal deposit as if a flash loan fee had just been repaid, artificially inflating the exchange rate every time anyone deposits, rather than only when a flash loan is actually repaid with a fee.

Impact: This has two consequences: 1. Since the exchange rate keeps rising on every deposit with no actual fee revenue backing it, it eventually overstates how much underlying token each `AssetToken` is redeemable for. When liquidity providers later try to `redeem`, the contract will not have enough underlying tokens to honor the inflated rate, causing legitimate withdrawals to revert. 2. The inflated rate also unfairly benefits earlier depositors and dilutes the real value calculation for liquidity providers who deposited during periods when a flash loan fee had genuinely just been collected, since deposits and real fee revenue are now indistinguishable in the rate calculation.

Proof of Concept: 1. Liquidity provider A deposits 1,000 `tokenA`. `exchangeRate` starts at its initial value (1e18). 2. Because `deposit` calls `updateExchangeRate(calculatedFee)` even though no flash loan fee was actually collected, the `exchangeRate` increases as though a fee had been paid. 3. Liquidity provider B repeats this deposit process several times. 4. Each of these deposits keeps inflating `exchangeRate`, even though the underlying token balance held by the `AssetToken` contract has only grown by the raw deposited amounts, not by any real fee revenue. 5. When liquidity providers attempt to `redeem` their `AssetToken` balance, the calculated underlying-token payout (based on the inflated `exchangeRate`) exceeds the actual token balance held by the contract, and the `redeem` call reverts due to insufficient balance.

Recommended Mitigation: Remove the `updateExchangeRate` call from the `deposit` function entirely. The exchange rate should only be updated when a flash loan fee is actually collected (i.e., inside `repay/_transferFundsBackAndUpdateBalance`), not on ordinary deposits.

```
1      function deposit(IERC20 token, uint256 amount) external
      revertIfZero(amount) revertIfNotAllowedToken(token) {
2          AssetToken assetToken = s_tokenToAssetToken[token];
3          uint256 exchangeRate = assetToken.getExchangeRate();
4          uint256 mintAmount = (amount * assetToken.
              EXCHANGE_RATE_PRECISION()) / exchangeRate;
5          emit Deposit(msg.sender, token, amount);
6          assetToken.mint(msg.sender, mintAmount);
7      -    uint256 calculatedFee = getCalculatedFee(token, amount);
8      -    assetToken.updateExchangeRate(calculatedFee);
9          token.safeTransferFrom(msg.sender, address(assetToken), amount)
              ;
10     }
```

[H-3] By calling a flashloan and then ThunderLoan::deposit instead of ThunderLoan::repay users can steal all funds from the protocol (Flash loan repayment verified only by raw token balance, not by which function returned the funds -> attacker mints AssetTokens with borrowed funds and redeems them, draining the pool)

Description: The `flashloan` function checks that funds were repaid by comparing the `AssetToken` contract's raw token balance before and after the borrower's callback executes — it does not require that repayment specifically go through the `repay` function.

```
1    function flashloan(address receiverAddress, IERC20 token, uint256
2        amount, bytes calldata params)
3        external
4        revertIfZero(amount)
5        revertIfNotAllowedToken(token)
6    {
7        AssetToken assetToken = s_tokenToAssetToken[token];
8        uint256 startingBalance = IERC20(token).balanceOf(address(
9            assetToken));
10
11        if (amount > startingBalance) {
12            revert ThunderLoan__NotEnoughLiquidity(amount,
13                startingBalance);
14        }
15
16        // ... transfers `amount` of `token` to `receiverAddress`,
17        // calls the receiver's callback ...
18
19        uint256 endingBalance = token.balanceOf(address(assetToken));
20        if (endingBalance < startingBalance + fee) {
21            revert ThunderLoan__NotPaidBack(startingBalance + fee,
22                endingBalance);
23        }
24    }
```

Since `flashloan` only checks the ending token balance of the `AssetToken` contract, a borrower can satisfy this check by calling `ThunderLoan::deposit(token, amount)` inside their flash loan callback instead of `ThunderLoan::repay`. `deposit` transfers the tokens to the same `AssetToken` contract address, which raises `endingBalance` enough to pass the check — but `deposit` also mints the caller brand-new `AssetToken` shares for that same amount, as if it were a legitimate liquidity deposit rather than a loan repayment.

Impact: The attacker effectively repays their flash loan using the protocol's own liquidity-provider accounting mechanism, receiving `AssetToken` shares for funds that were never actually theirs. They can then immediately call `redeem` to withdraw those funds (plus their proportional share of the pool's existing real liquidity) back out, permanently draining the protocol of all liquidity for that token.

Proof of Concept: 1. Attacker takes out a flashloan for `amount` of `tokenA` via `ThunderLoan::flashloan`. 2. Inside the flash loan callback (`executeOperation`), instead of approving and calling `ThunderLoan::repay`, the attacker calls `ThunderLoan::deposit(tokenA, amount + fee)`, transferring the borrowed funds (plus the small required fee) back to the `AssetToken` contract. 3. `flashloan`'s ending-balance check passes, since the `AssetToken` contract's raw token balance is now back at (or above) `startingBalance + fee` — but this happened via `deposit`, not `repay`. 4. The attacker now holds newly minted `AssetToken` shares corresponding to the “deposit” they just made, despite never having owned that capital. 5. The attacker calls `ThunderLoan::redeem`, burning those shares and withdrawing the underlying tokens — including the protocol's pre-existing liquidity — to their own wallet. 6. The attacker has now extracted funds from the protocol while nominally “repaying” the flash loan in full.

Recommended Mitigation: Track how much was borrowed in `s_currentlyFlashLoaning` (or similar) and require that `repay` — not `deposit` — is the only way to clear that outstanding amount. Alternatively, disallow calling `deposit` for a token while a flash loan for that token is currently active:

```
1      function deposit(IERC20 token, uint256 amount) external
      revertIfZero(amount) revertIfNotAllowedToken(token) {
2  +      if (s_currentlyFlashLoaning[token]) {
3  +          revert ThunderLoan__CannotDepositDuringFlashloan();
4  +      }
5      AssetToken assetToken = s_tokenToAssetToken[token];
6      ...
7  }
```

[H-4] getPriceOfOnePoolTokenInWeth uses the TSwap price which doesn't account for decimals, also fee precision is 18 decimals (Price and fee math assume all tokens use 18 decimals -> incorrect fee/price calculations for non-18-decimal tokens)

Description: `OracleUpgradeable::getPriceInWeth` reads a price directly from `ITSwapPool::getPriceOfOnePoolTokenInWeth()`, which returns a value scaled to 18 decimals (TSwap always quotes “price per 1e18 units” of the pool token, regardless of that token's actual decimal count). Separately, `ThunderLoan::getCalculatedFee` computes fees using `s_feePrecision`, which is hardcoded to `1e18`. Neither calculation accounts for the actual `decimals()` of the token being priced or borrowed.

```
1      function getPriceInWeth(address token) public view returns (uint256) {
2          address swapPoolOfToken = IPoolFactory(s_poolFactory).getPool(token);
3          return ITSwapPool(swapPoolOfToken).getPriceOfOnePoolTokenInWeth();
}
```

```
4      }
```

Impact: For any token that does not use 18 decimals (for example, USDC or USDT, which use 6 decimals), the price returned and the fee subsequently calculated from it will be off by orders of magnitude. This can cause the protocol to drastically undercharge or overcharge flash loan fees for such tokens, and can distort any downstream logic that relies on `getPriceInWeth/getCalculatedFee` being accurate.

Proof of Concept: 1. Assume `tokenA` uses 6 decimals (like USDC), and TSwap quotes its price per `1e18` pool-token units (i.e., assuming 18 decimals). 2. A user requests a flash loan of `1_000_000` (i.e., `1.0 tokenA`, in its native 6-decimal representation). 3. `getCalculatedFee` uses this raw amount together with the 18-decimals-per-unit price and `s_feePrecision = 1e18` to compute the fee, effectively treating `1_000_000` as a vanishingly small fraction of one token rather than a whole unit. 4. The resulting fee is far smaller than the intended 0.3%, since the decimal mismatch is not corrected anywhere in the calculation.

Recommended Mitigation: Normalize all price and fee calculations to a consistent decimal precision by reading and adjusting for each token's actual `decimals()` (via `IERC20Metadata`) before using it in fee or price math, rather than assuming every token uses 18 decimals.

Medium

[M-1] Centralization risk for trusted owners (Single `onlyOwner` address controls token allow-listing and contract upgrades -> a malicious or compromised owner can brick redemptions or rewrite protocol logic entirely)

Description: `ThunderLoan.sol` grants its `owner` unilateral, unrestricted control over two highly consequential actions: which tokens are allowed in the protocol, and what code the protocol's upgradeable proxy points to. There is no multisig requirement, timelock, or governance process gating either capability.

Instances (2):

```
1 File: src/protocol/ThunderLoan.sol
2
3 223:     function setAllowedToken(IERC20 token, bool allowed) external
         onlyOwner returns (AssetToken) {
4
5 261:     function _authorizeUpgrade(address newImplementation) internal
         override onlyOwner { }
```

Impact: Contracts have owners with privileged rights to perform admin tasks and need to be trusted to not perform malicious updates or drain funds.

Contralized owners can brick redemptions by disapproving of a specific token If the owner calls `setAllowedToken(token, false)` for a token that liquidity providers currently hold `AssetToken` shares for, those liquidity providers can be prevented from performing new deposits or flash loans for that token — and depending on how redemption logic checks the allow-list, existing liquidity providers could also be blocked from redeeming their shares, effectively locking their funds until the owner reverses the change (or indefinitely, if they do not).

Proof of Concept: 1. Liquidity provider deposits `tokenA` and receives `AssetToken` shares. 2. Owner calls `setAllowedToken(tokenA, false)`. 3. Liquidity provider attempts to `redeem` their shares for `tokenA` and, depending on whether `redeem` is also gated by `revertIfNotAllowedToken`, either fails outright or the token becomes unusable for any further protocol interaction, leaving the provider's funds stranded. 4. Separately, the same owner key (or an attacker who compromises it) can call `_authorizeUpgrade` to point the proxy at an arbitrary new implementation, rewriting the protocol's entire logic — including redirecting all deposited funds — at will.

Recommended Mitigation: Route `setAllowedToken` and `_authorizeUpgrade` through a multisig wallet and/or a timelocked governance process rather than a single EOA-controlled `onlyOwner` modifier, so that no single compromised or malicious key can unilaterally disable a token or rewrite the protocol's logic without a delay the community can react to.

[M-2] Using TSwap as price oracle leads to price and oracle manipulation attacks (Flash-loanable capital used to manipulate a low-liquidity AMM's spot price -> attacker distorts ThunderLoan's fee calculation to their advantage)

Description: The TSwap protocol is a constant product formula based AMM (automated market maker). The price of a token is determined by how many reserves are on either side of the pool. Because of this, it is easy for malicious users to manipulate the price of a token by buying or selling a large amount of the token in the same transaction, essentially ignoring protocol fees.

Impact: Liquidity providers will drastically reduced fees for providing liquidity.

Proof of Concept:

The following all happens in 1 transaction.

1. User takes a flash loan from [ThunderLoan](#) for 1000 [tokenA](#). They are charged the original fee [fee1](#). During the flash loan, they do the following:
 1. User sells 1000 [tokenA](#), tanking the price.
 2. Instead of repaying right away, the user takes out another flash loan for another 1000 [tokenA](#).
 1. Due to the fact that the way [ThunderLoan](#) calculates price based on the [TSwapPool](#) this second flash loan is substantially cheaper.

```
1 function getPriceInWeth(address token) public view returns (
    uint256) {
2     address swapPoolOfToken = IPoolFactory(s_poolFactory).
        getPool(token);
3     @> return ITSwapPool(swapPoolOfToken).
        getPriceOfOnePoolTokenInWeth();
4 }
```

3. The user then repays the first flash loan, and then repays the second flash loan.

I have created a proof of code located in my [audit-data](#) folder. It is too large to include here.

Recommended Mitigation: Consider using a different price oracle mechanism, like a Chainlink price feed with a Uniswap TWAP fallback oracle.

[M-4] Fee on transfer, rebase, etc (Protocol assumes transferred amount always equals received amount -> accounting breaks for fee-on-transfer and rebasing tokens)

Description: Throughout [ThunderLoan.sol](#), the amount of a token that is transferred (via [safeTransfer/safeTransferFrom](#)) is assumed to be the exact amount that the receiving contract's balance increases by. This assumption does not hold for two categories of ERC20 tokens: - **Fee-on-transfer tokens** — deduct a fee during the transfer itself, so the recipient receives less than the amount specified. - **Rebasing tokens** — automatically adjust every holder's balance over time (up or down) independent of any transfer, meaning a stored balance or exchange-rate calculation can silently drift out of sync with the token's actual live balance.

Neither [deposit](#), [flashloan](#), [repay](#), nor the [AssetToken](#) exchange-rate accounting verify the actual post-transfer balance change; they all assume the nominal [amount](#) parameter is exactly what was received.

Impact: For a fee-on-transfer token, [AssetToken](#)'s recorded balance/exchange-rate math will overstate how much of the underlying token actually exists in the contract, since less arrives than was nominally transferred. This can cause the same class of “promises more than it holds” failure described

in [H-2], where liquidity providers are unable to redeem their full expected balance. For rebasing tokens, the protocol's internal exchange-rate accounting can drift arbitrarily far from the token's real balance over time, with no mechanism to reconcile the two.

Proof of Concept: 1. Owner calls `setAllowedToken` for a fee-on-transfer token, `tokenFOT`, which deducts 1% on every transfer. 2. A liquidity provider calls `deposit(tokenFOT, 1000)`. `ThunderLoan` mints `AssetToken` shares based on the full 1000, but due to the transfer fee, the `AssetToken` contract's balance only increases by 990. 3. The protocol's internal accounting now believes it holds 1000 units of `tokenFOT` backing the minted shares, while it actually holds only 990 — a permanent, compounding shortfall as more such deposits/withdrawals occur.

Recommended Mitigation: Either explicitly disallow fee-on-transfer and rebasing tokens via the allow-list, or measure the recipient's actual balance before and after each transfer and use that measured delta (rather than the nominal `amount` parameter) for all subsequent accounting.

Low

[L-1] Empty Function Body - Consider commenting why (Uncommented empty override -> reviewers cannot tell whether the empty body is intentional or a missing implementation)

Description: `_authorizeUpgrade` is required by the UUPS upgrade pattern to be overridden, and here it is left with an empty body, relying solely on the `onlyOwner` modifier to restrict who may authorize an upgrade. An empty function body with no explanatory comment is difficult to distinguish, on review, from an accidentally-incomplete implementation.

Instances (1):

```
1 File: src/protocol/ThunderLoan.sol
2
3 261:     function _authorizeUpgrade(address newImplementation) internal
      override onlyOwner { }
```

Impact: Low — this does not introduce a vulnerability on its own (the `onlyOwner` modifier is sufficient for this function's intended purpose), but the lack of a comment increases the chance that a future maintainer or auditor misreads the empty body as a bug, or that a future edit accidentally removes the `onlyOwner` check without anyone noticing the significance of the now-empty body.

Proof of Concept: N/A — this is a code-clarity observation rather than an exploitable condition.

Recommended Mitigation: Add a short comment explaining that the empty body is intentional and that `onlyOwner` is the sole and sufficient access control for this function:

```
1 + // Intentionally empty: onlyOwner is sufficient access control for
   // authorizing upgrades.
2   function _authorizeUpgrade(address newImplementation) internal
   override onlyOwner { }
```

[L-2] Initializers could be front-run (Unrestricted first-caller of `initialize` -> attacker who wins the race can set themselves as owner or point the oracle at a malicious factory)

Description: Initializers could be front-run, allowing an attacker to either set their own values, take ownership of the contract, and in the best case forcing a re-deployment.

Instances (6):

```
1 File: src/protocol/OracleUpgradeable.sol
2
3 11:     function __Oracle_init(address poolFactoryAddress) internal
   onlyInitializing { }
```

```
1 File: src/protocol/ThunderLoan.sol
2
3 138:     function initialize(address tswapAddress) external initializer
   {
4
5 138:     function initialize(address tswapAddress) external initializer
   {
6
7 139:         __Ownable_init();
8
9 140:         __UUPSUpgradeable_init();
10
11 141:         __Oracle_init(tswapAddress);
```

Impact: The `initializer` modifier guarantees `initialize` can only succeed once, but places no restriction on *who* is allowed to be the caller that wins that one opportunity. If an attacker's call to `initialize` is mined before the legitimate deployer's, the attacker becomes the contract's `owner` and can set `s_poolFactory` to an address they control, corrupting all subsequent pricing via `OracleUpgradeable::getPriceInWeth`, and gaining upgrade authority over the entire protocol via `_authorizeUpgrade`.

Proof of Concept: 1. The protocol deployer submits a transaction calling `initialize(realTSwapFactoryAddress)`. 2. While that transaction is pending in the mempool, an attacker observes it and submits their own `initialize(attackerControlledAddress)` call with a higher gas price. 3. The attacker's transaction is mined first, permanently setting them as `owner` and

`s_poolFactory` to their own contract. 4. The deployer's original transaction then reverts, since `initializer` blocks a second successful call.

Recommended Mitigation: This cannot be fully mitigated at the code level, since `initializer` already enforces “exactly once” — the remaining risk is purely about timing. Mitigate operationally by bundling contract deployment and initialization into a single atomic transaction (e.g., within a deployment script), removing the window during which a separate `initialize` call could be front-run.

[L-3] Missing critical event emissions (No event emitted when `s_flashLoanFee` changes -> off-chain systems and users cannot detect fee changes)

Description: When the `ThunderLoan : : s_flashLoanFee` is updated, there is no event emitted.

Impact: Off-chain monitoring tools, frontends, and integrators have no reliable way to detect when the protocol's flash loan fee changes, other than polling contract state directly. This reduces transparency and could allow a fee change to go unnoticed by users or dependent protocols until they are unexpectedly charged a different rate.

Proof of Concept: Call `updateFlashLoanFee(newFee)` as the owner and observe that no event is emitted in the transaction logs, despite `s_flashLoanFee` changing in storage.

Recommended Mitigation: Emit an event when the `ThunderLoan : : s_flashLoanFee` is updated.

```
1 +     event FlashLoanFeeUpdated(uint256 newFee);
2 .
3 .
4 .
5     function updateFlashLoanFee(uint256 newFee) external onlyOwner {
6         if (newFee > s_feePrecision) {
7             revert ThunderLoan__BadNewFee();
8         }
9         s_flashLoanFee = newFee;
10 +     emit FlashLoanFeeUpdated(newFee);
11 }
```

Informational

[I-1] Poor Test Coverage (Coverage below acceptable threshold across core contracts -> undiscovered edge cases and regressions are more likely to reach production)

Description: Running the project's test suite with coverage reporting shows several core contracts falling well short of a reasonable coverage threshold, particularly around branch coverage in `ThunderLoan.sol`.

1	Running tests...			
2	File		% Lines	% Statements
3	% Branches	% Funcs		
4	src/protocol/AssetToken.sol		70.00% (7/10)	76.92% (10/13)
5	src/protocol/OracleUpgradeable.sol		100.00% (6/6)	100.00% (9/9)
6	src/protocol/ThunderLoan.sol		64.52% (40/62)	68.35% (54/79)

Impact: Low test coverage, especially the 37.50% branch coverage in `ThunderLoan.sol`, means many conditional paths (including error/revert cases) are never exercised by the existing test suite. This increases the likelihood that a future change introduces a regression that goes undetected, and it also means some of the findings in this report (such as [H-2] and [H-3]) were not caught by the project's own tests before this audit.

Proof of Concept: N/A — see the coverage table above, generated via the project's coverage tooling.

Recommended Mitigation: Aim to get test coverage up to over 90% for all files, with particular attention to branch coverage for conditional logic in `ThunderLoan.sol`.

[I-2] Not using `__gap[50]` for future storage collision mitigation (No reserved storage slots in upgradeable base contracts -> adding new state variables to a parent contract in a future upgrade will shift and corrupt child contract storage)

Description: None of the upgradeable base contracts in scope (e.g., `OracleUpgradeable.sol`) reserve any unused storage slots via a `uint256[50] private __gap;`-style array, a standard OpenZeppelin convention for upgradeable base contracts.

Impact: If a future version of `OracleUpgradeable` (or any other inherited upgradeable base contract) needs to add a new state variable, doing so will shift the storage slot positions of every variable

declared in the contracts that inherit from it (such as `ThunderLoan.sol`), causing exactly the kind of storage collision described in [H-1] — except triggered by a change to a parent contract rather than the main contract itself, making it easier to overlook.

Proof of Concept: N/A — this is a preventative/architectural recommendation for future upgrade safety rather than a currently-exploitable condition.

Recommended Mitigation: Add a reserved storage gap to each upgradeable base contract, following OpenZeppelin's convention:

```
1 contract OracleUpgradeable is Initializable {
2     address private s_poolFactory;
3 +   uint256[49] private __gap;
4     ...
5 }
```

[I-3] Different decimals may cause confusion. ie: AssetToken has 18, but asset has 6 (Fixed 18-decimal assumption in AssetToken regardless of underlying asset's actual decimals -> confusing off-chain displays and potential integration bugs)

Description: `AssetToken` is implemented assuming 18 decimals (matching `EXCHANGE_RATE_PRECISION = 1e18`), but the underlying token it wraps may use a different decimal count — for example, USDC uses 6 decimals. `AssetToken` does not appear to align its own `decimals()` with the underlying asset's `decimals()`.

Impact: This is primarily a usability/integration risk rather than a direct fund-loss vulnerability: frontends, wallets, and integrating contracts that naively assume an `AssetToken`'s decimals match its underlying asset's decimals could display or calculate incorrect amounts, potentially leading users to misjudge the value of their holdings or causing integration bugs in third-party tooling.

Proof of Concept: Deploy an `AssetToken` wrapping a 6-decimal token (e.g., USDC) and compare `AssetToken.decimals()` against the underlying token's `decimals()` — they will not match, since `AssetToken` is hardcoded to 18-decimal-style precision internally.

Recommended Mitigation: Consider having `AssetToken : decimals()` return the same value as the underlying asset's `decimals()`, and clearly document anywhere decimals are assumed within the contract, to reduce the risk of downstream confusion.

[I-4] Doesn't follow <https://eips.ethereum.org/EIPS/eip-3156> (Custom flash loan interface instead of the standard -> reduced composability with tooling and aggregators built against EIP-3156)

Description: EIP-3156 defines a standardized flash loan interface ([IERC3156FlashLender](#) / [IERC3156FlashBorrower](#)) that many DeFi tools, aggregators, and integrators expect flash loan providers to implement. [ThunderLoan.sol](#) implements its own custom [flashloan](#) / [IFlashLoanReceiver](#) interface instead of conforming to this standard.

Impact: Low — the protocol functions correctly on its own, but third-party tooling, aggregators, or borrower contracts built generically against EIP-3156 will not be able to interact with [ThunderLoan](#) without protocol-specific integration work, reducing composability within the broader DeFi ecosystem.

Proof of Concept: N/A — this is a standards-conformance observation rather than an exploitable condition.

Recommended Mitigation: Consider implementing the standard EIP-3156 interfaces ([IERC3156FlashLender](#) , [IERC3156FlashBorrower](#)) alongside or instead of the current custom interface, to maximize compatibility with existing flash loan tooling and integrators.

Gas**[GAS-1] Using bools for storage incurs overhead**

Use [uint256](#)(1) and [uint256](#)(2) for true/false to avoid a [Gwarmaccess](#) (100 gas), and to avoid [Gsset](#) (20000 gas) when changing from 'false' to 'true', after having been 'true' in the past. See source.

Instances (1):

```
1 File: src/protocol/ThunderLoan.sol
2
3 98:      mapping(IERC20 token => bool currentlyFlashLoaning) private
      s_currentlyFlashLoaning;
```

[GAS-2] Using private rather than public for constants, saves gas

If needed, the values can be read from the verified contract source code, or if there are multiple values there can be a single getter function that returns a tuple of the values of all currently-public constants. Saves **3406-3606 gas** in deployment gas due to the compiler not having to create non-payable getter

functions for deployment calldata, not having to store the bytes of the value outside of where it's used, and not adding another entry to the method ID table

Instances (3):

```
1 File: src/protocol/AssetToken.sol
2
3 25:      uint256 public constant EXCHANGE_RATE_PRECISION = 1e18;
```

```
1 File: src/protocol/ThunderLoan.sol
2
3 95:      uint256 public constant FLASH_LOAN_FEE = 3e15; // 0.3% ETH fee
4
5 96:      uint256 public constant FEE_PRECISION = 1e18;
```

[GAS-3] Unnecessary SLOAD when logging new exchange rate

In `AssetToken::updateExchangeRate`, after writing the `newExchangeRate` to storage, the function reads the value from storage again to log it in the `ExchangeRateUpdated` event.

To avoid the unnecessary SLOAD, you can log the value of `newExchangeRate`.

```
1   s_exchangeRate = newExchangeRate;
2 - emit ExchangeRateUpdated(s_exchangeRate);
3 + emit ExchangeRateUpdated(newExchangeRate);
```